Requirements Gathering Process & System Specifications

Data Engineering Graduation Project — Clinical Data Pipeline & Analytics Platform

Domain / Scale: Healthcare Clinical Analytics (~1,000,000 Records / Cells)	**Target Architecture:** Medallion (Bronze → Silver → Gold) & Star Schema
Key Technologies: Databricks, PySpark, Delta Lake, DBT, PostgreSQL, Power BI	**Team Composition:** 4 Tracks (Data Architect, Analytics Engineer, Platform Engineer, BI Developer)

1. Step 1: Identify Stakeholders

1.1 Executive Summary & Stakeholder Classification

The successful delivery of the Clinical Data Pipeline and Executive Intelligence Platform requires clear alignment across both technical execution teams and organizational decision-makers.

Internal Stakeholders (Engineering & Delivery Team):

- **Track 1: Data Architect (Member 1):** Responsible for Silver layer quality verification, schema enforcement, surrogate key creation, and Gold dimension modeling.
- **Track 2: Analytics Engineer — Dr. Aya Elsaify (Member 2):** Responsible for core Gold fact table engineering, clinical KPI aggregation (ICU Length of Stay, mortality rates, readmission flags), and Delta Lake storage optimization (OPTIMIZE / ZORDER BY).
- **Track 3: Platform Engineer (Member 3):** Responsible for Databricks multi-task workflow orchestration, automated retry policies, PostgreSQL database sink management, and GitHub repository governance.
- **Track 4: BI Developer (Member 4):** Responsible for Power BI semantic star-schema modeling, DAX measure creation, interactive executive dashboard design, and graduation presentation deck assembly.

External Stakeholders & Business Owners:

- **DEPI Academic & Evaluation Panel:** Primary evaluators assessing architectural rigor, data modeling correctness, data quality standards, and platform reliability.
- **Clinical Operations & Healthcare Executives:** End-users relying on clinical dashboards for ICU bed utilization tracking, surgical outcome evaluation, and patient mortality analysis.

2. Step 2: Plan the Approach

2.1 Data Scale & Architectural Methodology

The project processes a clinical dataset containing ~1,000,000 data cells/records. To guarantee scalability, reliability, and clear team boundaries, the architecture adopts the Medallion Paradigm (Bronze → Silver → Gold) integrated with dbt (Data Build Tool) transformation patterns.

2.2 Track Allocation & Git Merge Isolation

To prevent Git merge conflicts and blocking dependencies across the 4 members, strict boundary handoffs have been established:

Track Handoff Specifications:

- **Handoff A (Member 1 → Member 2):** Member 1 delivers finalized Silver Delta tables and Gold Dimension tables (dim_patients, dim_physicians, dim_procedures, dim_diagnoses, dim_date) with validated surrogate keys.
- **Handoff B (Member 2 → Member 3):** Member 2 delivers fully aggregated, optimized Gold Fact tables (fact_icu_stays, fact_surgical_encounters, fact_vitals_labs) and clinical metric views.
- **Handoff C (Member 3 → Member 4):** Member 3 delivers automated Databricks Multi-task Workflows, production PostgreSQL database synchronization, and clean main repository branches.
- **Handoff D (Member 4 → Evaluation):** Member 4 connects Power BI to Gold views/PostgreSQL to deliver the final semantic model, interactive dashboards, and executive presentation.

3. Step 3: Conduct Elicitation

3.1 Domain Challenges & Elicited Clinical Requirements

Through structured elicitation with domain requirements, the following key operational and clinical pain points were identified and mapped to technical solutions:

Key Clinical Pain Points & Pipeline Solutions:

- **Unstructured & Noisy Clinical Feeds:** Raw EHR data contains missing timestamps and null values. Solution: Implement strict Silver quality checks and standardize datetime formats across all clinical events.
- **ICU Capacity & Length of Stay (LOS) Bottlenecks:** Hospital management lacks visibility into bed turnover and prolonged stays. Solution: Engineer dedicated PySpark aggregation models for ICU Length of Stay (LOS) and 30-day readmission flags.
- **Surgical Workflow Variances:** Operating room scheduling suffers from unexpected procedure delays. Solution: Calculate surgical duration variances by comparing scheduled vs. actual procedure times in Gold fact tables.

- **High Query Latency on Large Datasets (~1M Rows):** Unoptimized clinical queries stall executive reports. Solution: Apply Delta Lake OPTIMIZE and ZORDER BY indexing on patient ID, admission date, and facility location.

4. Step 4: Document Observations & Findings

4.1 System Requirements Categorization

All gathered observations have been categorized into Functional Requirements (FR), Non-Functional Requirements (NFR), and System Constraints.

Functional Requirements (FR):

- **FR-01 (Data Ingestion & Staging):** The pipeline must ingest raw CSV/EHR data into Bronze staging views (stg_customers / stg_orders structure adaptors).
- **FR-02 (Silver Cleaning & Audit):** Silver transformation scripts must filter nulls, enforce datatypes, and remove duplicate clinical records.
- **FR-03 (Dimension & Fact Modeling):** PySpark models must build Star Schema dimensions with surrogate keys and fact tables joining clinical encounters.
- **FR-04 (Clinical KPI Aggregation):** Gold layers must calculate ICU LOS, mortality rates, readmission flags, and procedure duration variances.
- **FR-05 (Workflow Automation):** Databricks Workflows must orchestrate Multi-task Jobs (Bronze → Silver → Gold) sequentially with retry alerts.
- **FR-06 (External Sync & BI Dashboarding):** Gold tables must export to PostgreSQL and power dynamic Power BI visuals with drill-through filters.

Non-Functional Requirements (NFR) & Technical Constraints:

- **NFR-01 (Performance & Latency):** Dashboard query response time must be under 2 seconds for ~1M clinical records using ZORDER BY optimizations.
- **NFR-02 (Data Quality & Integrity):** Zero duplicate primary keys allowed in Gold dimensions; 100% referential integrity enforced across fact foreign keys.
- **Constraint-01 (Tech Stack Alignment):** Execution is strictly restricted to Databricks, PySpark, Delta Lake, dbt, PostgreSQL, Power BI, and GitHub.

5. Step 5: Analyze & Prioritize (MoSCoW Framework)

5.1 MoSCoW Prioritization Matrix

To guarantee on-time graduation project delivery, requirements are prioritized using MoSCoW:

MoSCoW Classifications:

- **Must Have (Core Requirements):** Silver data cleaning, Gold Dimension and Fact tables, core Clinical KPIs (ICU LOS, mortality), Power BI Star Schema, and basic GitHub repo setup.
- **Should Have (High Value Additions):** Databricks Workflows multi-task orchestration, Delta Lake OPTIMIZE & ZORDER BY indexing, automated email/alert notifications, and dbt data testing.
- **Could Have (Enhanced Capabilities):** Automated PostgreSQL sink sync out of Databricks, advanced Power BI drill-through pages, and automated CI/CD GitHub Actions.
- **Won't Have (Out of Scope for Initial Release):** Real-time streaming ingestion (Kafka/Structured Streaming) and complex machine learning predictive modeling.

6. Step 6: Verify & Validate

6.1 Quality Assurance & Verification Protocols

Prior to engineering release, all transformations and data models must undergo strict validation:

Verification Workflows:

- **ERD & Schema Alignment:** Walkthrough reviews verifying all surrogate primary keys in dim_patients, dim_physicians, etc., align 100% with foreign keys in fact tables.
- **Automated Data Testing (dbt & PySpark):** Enforce not_null, unique, and expression_is_true assertions across all staging and fact tables (similar to staging.yml / facts.yml tests).
- **Peer Review & Pull Requests:** No code is merged into main without Member 3 (Platform Engineer) review, resolving branch conflicts, and verifying execution logs.

7. Step 7: Manage Changes & Traceability

7.1 Traceability Matrix & Change Control

To handle scope changes gracefully and maintain complete data lineage throughout implementation:

Governance Protocols:

- **Traceability Matrix (RTM):** Maps every raw clinical attribute from Bronze sources through Silver cleaning, Gold dimension/fact joins, down to specific DAX measures in Power BI.
- **Git Branching Strategy:** Feature-based branching (e.g., feature/silver-audit, feature/gold-facts, feature/powerbi-model) isolated per team member.
- **Change Control Process:** Any schema or metric definition modification requires formal agreement across impacted tracks before updating dbt project configurations or PySpark scripts.

8. Team Responsibilities & Deliverables Summary Matrix

Role / Track	Member	Primary Responsibilities	Key Deliverables
Data Architect	Member 1	Silver quality checks, schema validation, Gold dimension modeling	dim_patients, dim_physicians, ERD alignment
Analytics Engineer	Dr. Aya (Member 2)	Fact table engineering, Clinical KPIs (LOS, mortality), Delta OPTIMIZE	fact_icu_stays, fact_vitals, ZORDER BY index
Platform Engineer	Member 3	Databricks Workflows, PostgreSQL sync, Git repo governance	Multi-task Jobs, GitHub README.md, PRs
BI Developer	Member 4	Power BI Star Schema, DAX metrics, Executive dashboards	Power BI report, Graduation presentation deck